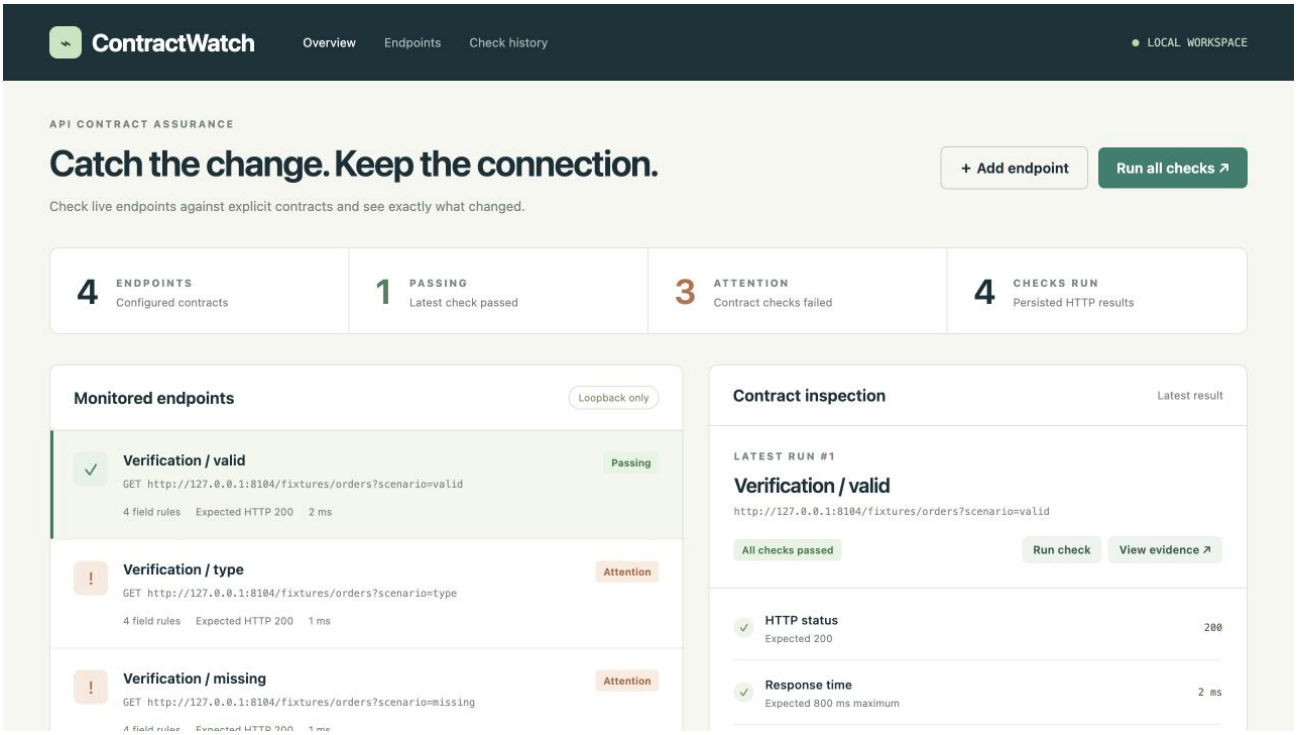
PROJECT 04 / API & SYSTEM INTEGRATION

ContractWatch

API contract checks that explain integration failures

Independent software project | Owner: Ismail Habib | Created September 2026



Actual application capture. Any example records are labeled verification inputs.

The problem

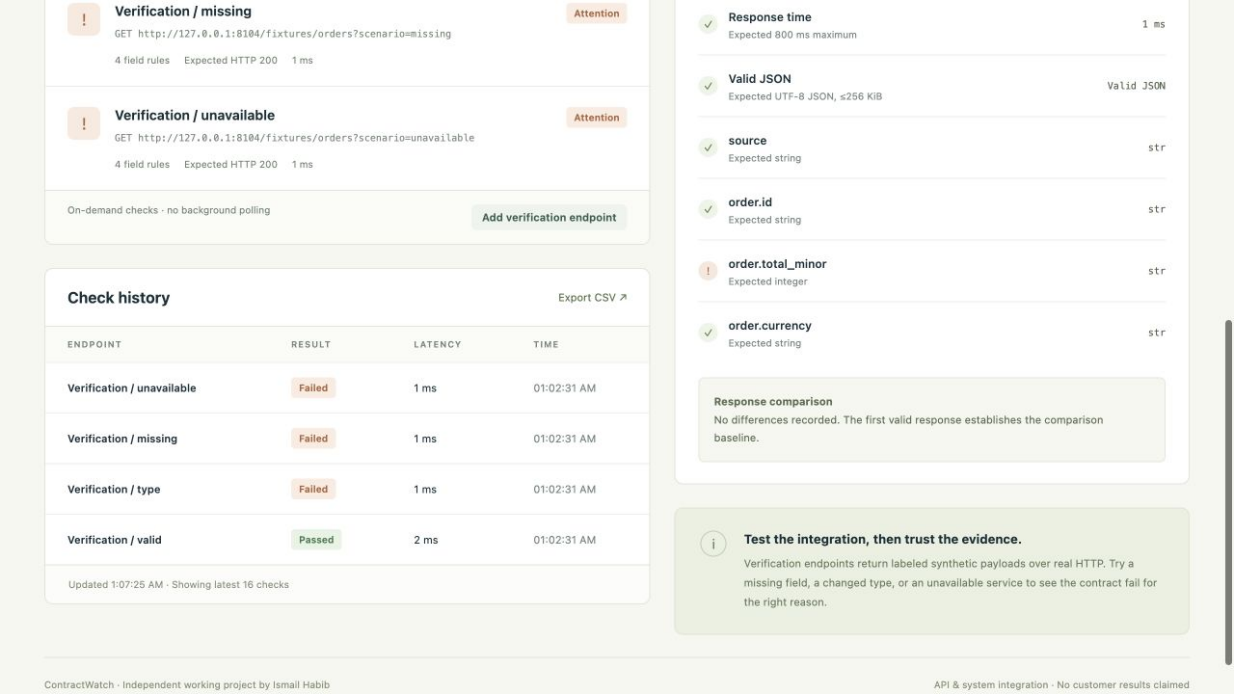
An endpoint can return HTTP 200 while removing a required field or changing a numeric value into text. A basic availability check misses these failures. Developers need explicit response expectations, meaningful diagnostics and retained evidence so they can identify which part of an integration changed and investigate it without guessing.

What the project delivers

- Status, latency and required typed-field contracts over real HTTP
- Persisted JSON responses and bounded response-path differences
- Inspectable failures, CSV export and command-line check execution
- Separate contract versions and disclosed local verification scenarios

ENGINEERING & EVIDENCE

Built to be inspected.



Detailed failure evidence connects a deliberately invalid verification response to its exact failed requirement and persisted check result.

What was verified

Eleven automated tests passed on 28 September 2026. Real loopback requests verified successful contracts, a missing required ID, a numeric field changed to text, HTTP 503, invalid JSON and a measured one-second response exceeding its latency limit. Tests also verified changed response paths, boolean/integer distinctions, array lookup, request restrictions, version preservation and database persistence. The displayed endpoint outcomes come from labeled synthetic verification inputs. These checks establish behavior for those cases, not production monitoring history, API compatibility or customer outcomes.

How it works

Define endpoint contract > Request real JSON > Evaluate and compare > Inspect stored evidence

Current scope

This independent local tool runs checks on demand; it does not provide scheduled monitoring, automatic alert delivery, hosted uptime tracking or a distributed runner. Its path/type rules are narrower than full JSON Schema. HTTP requests are bounded by time and size limits, and response bodies remain in the local database. Remote endpoints require opt-in. The optional bearer token is shared across configured endpoints, so authenticated use requires trusted destinations. Public deployment needs access controls, retention decisions and further operational hardening.

Included: source code, setup guide, tests, screenshots, editable case study and Malt copy.
MIT License | Copyright 2026 Ismail Habib.